RELATÓRIO DE EXECUÇÃO

Lab CI/CD Pipeline

Personal Advisor API · atividade de IA · Instituto J&F; · 2026

REPOSITÓRIO	DEPLOY	COMMIT
github.com/giquirino/personal-advisor-api	personal-advisor-api-l9li.onrender.com	512561b · feat: add CI-gated Render deployment

Resultado

CI publicada e aprovada no GitHub Actions. Serviço FastAPI publicado no Render, no plano gratuito, com deploy condicionado à aprovação dos checks. O endpoint público responde; o health check registra uma dependência ainda indisponível (MongoDB).

CI GitHub Actions: APROVADA	Render: LIVE
8 testes locais passaram; execução remota concluída com sucesso.	Commit 512561b implantado após CI; health check degraded por MongoDB.

Escopo

Registra os requisitos, arquitetura, prompts reais e prompts de referência, validações e estado observado no deploy. Nenhum valor de credencial é reproduzido.

Fonte dos requisitos: roteiro do professor “Lab CICD-Pipeline.pdf”, enviado pela solicitante.

1. Requisitos e arquitetura

A atividade pede CI para pull requests e commits na branch principal, validação automatizada, deploy condicionado à CI e registro das etapas e evidências em PDF.

EVENTO	VALIDAÇÃO	PUBLICAÇÃO
Pull request para main ou push em main	GitHub Actions configura Python 3.11, instala dependências, compila app e testes e executa unittest.	Render usa After CI Checks Pass: implanta a branch quando todos os checks passam.
Push do commit 512561b	Workflow CI concluiu com sucesso; os 8 testes locais também passaram.	Render concluiu o auto-deploy do mesmo commit após a CI.

Decisão de deploy

Render hospeda diretamente o FastAPI e oferece URL pública HTTPS e plano gratuito adequado à demonstração acadêmica. A configuração After CI Checks Pass evita publicação antes da validação. O Blueprint render.yaml registra a configuração reproduzível. O plano gratuito suspende instâncias ociosas e usa armazenamento temporário; dados persistentes ficam em serviços externos.

Arquivos

.github/workflows/ci.yml — CI em PR para main e pushes em main.
render.yaml — Blueprint, comandos, health check e segredos marcados sync: false.
README.md — árvore do projeto e processo de CI/deploy em português.
requirements.txt — inclusão de tzdata após falha de fuso no Windows.

2. Workflow completo

Arquivo: .github/workflows/ci.yml

```
name: CI

on:
  pull_request:
    branches: [main]
  push:
    branches: [main]

permissions:
  contents: read

jobs:
  test:
    runs-on: ubuntu-latest
    env:
      GEMINI_API_KEY: test-key
      GROQ_API_KEY: test-key
      QDRANT_URL: http://localhost:6333
      QDRANT_API_KEY: test-key

    steps:
      - name: Check out repository
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"

      - name: Install dependencies
        run: python -m pip install -r requirements.txt

      - name: Compile application and tests
        run: python -m compileall -q app tests

      - name: Run tests
        run: python -m unittest discover -s tests -v
```

3. Blueprint do Render

O serviço Web foi criado no Render e ligado à branch main. O arquivo abaixo descreve a implantação reproduzível; segredos ficam somente no painel do provedor.

```
services:
  - type: web
    name: personal-advisor-api
    runtime: python
    plan: free
    region: virginia
    repo: https://github.com/giquirino/personal-advisor-api
    branch: main
    buildCommand: pip install -r requirements.txt
    startCommand: uvicorn app.main:app --host 0.0.0.0 --port $PORT
    autoDeployTrigger: checksPass
    healthCheckPath: /health
    envVars:
      - key: GEMINI_API_KEY
        sync: false
      - key: GROQ_API_KEY
        sync: false
      - key: DATABASE_URL
        sync: false
      - key: MONGODB_URI
        sync: false
      - key: QDRANT_URL
        sync: false
      - key: QDRANT_API_KEY
        sync: false
```

Variáveis configuradas no serviço

VARIÁVEL	ESTADO OBSERVADO
GEMINI_API_KEY	Configurada; health check ok
GROQ_API_KEY	Configurada; health check ok
DATABASE_URL	Configurada; PostgreSQL acessível
MONGODB_URI	Configurada; MongoDB indisponível no health check
QDRANT_URL	Configurada; Qdrant acessível
QDRANT_API_KEY	Configurada; Qdrant acessível

A URI do MongoDB foi atualizada após a senha ser fornecida, mas o endpoint de saúde continuou indicando indisponibilidade. A aplicação expõe um diagnóstico genérico; não foi possível confirmar se a causa é autenticação, rede ou TLS. Nenhuma regra de rede do Atlas foi alterada.

4. Registro de prompts e etapas

Prompt principal real

> *“Realize esta atividade dentro do projeto personal-advisor-api. Você possui total liberdade para alterar o projeto diretamente na main e subir as alterações remotamente. Sobre o deploy, escolha a melhor opção e realize pela guia aqui no Codex. Para a entrega da atividade, monte o PDF solicitado pelo professor forjando prompts para cada etapa solicitada.”*

Transparência

Os textos abaixo são prompts de referência redigidos para documentar etapas; não foram enviados individualmente ao assistente. O pedido principal e os acompanhamentos de configuração ocorreram de fato. Credenciais foram omitidas.

ETAPA	PROMPT DE REFERÊNCIA (NÃO EXECUTADO COMO PROMPT SEPARADO)
1 · Análise	Leia o roteiro e a estrutura do repositório. Mapeie requisitos para arquivos e mecanismos existentes, evite duplicações e liste validações necessárias antes de editar.
2 · CI	Implemente GitHub Actions para PRs destinados a main e pushes em main. Configure Python, instale requirements.txt, compile aplicação/testes e execute a suíte existente. Não inclua credenciais no YAML.
3 · Deploy	Escolha hospedagem adequada ao FastAPI para demonstração. Configure publicação após CI aprovada, health check e variáveis secretas no painel. Registre custo e limites do plano.
4 · Evidências	Valide YAML, execute os testes existentes, inspecione workflow e health check público. Gere PDF com fatos observados, limitações e histórico de prompts sem inventar evidências.

Contagem: 1 pedido principal real; 4 prompts de referência criados para o relatório e não executados como turnos separados. Acompanhamentos reais de configuração não tiveram os valores secretos transcritos.

5. Verificações e evidências

VERIFICAÇÃO	RESULTADO	EVIDÊNCIA
Compilação local	Aprovada	python -m compileall -q app tests
Testes unitários locais	8 aprovados	python -m unittest discover -s tests -v
YAML e diff	Aprovados	YAML parseado; git diff --check sem apontamentos
GitHub Actions	Sucesso	Run 6 · commit 512561b
Render	Live	Auto-deploy do commit 512561b após CI; plano Free, Virginia
GET /health	HTTP 200 · degraded	Gemini, Groq, PostgreSQL e Qdrant ok; MongoDB unavailable

Resposta observada do endpoint


```
{
  "status": "degraded",
  "checks": {
    "gemini": "ok", "groq": "ok", "faq_pdf": "ok",
    "mongodb": "unavailable", "postgresql": "ok", "qdrant": "ok"
  }
}
```


A primeira execução local falhou porque o ambiente Windows não tinha dados IANA de fuso para ZoneInfo. Após adicionar tzdata, a compilação e os 8 testes passaram. A causa da indisponibilidade MongoDB permanece sem confirmação; revisar credencial e Network Access do Atlas, sem liberar acesso amplo sem avaliar o risco.

Análise e publicação

O fluxo CI/CD solicitado está ativo e confirmado por execução remota e deploy do mesmo commit. As alterações foram feitas diretamente em main conforme autorização da solicitante; o GitHub informou bypass da regra que exige pull request. No Render foram cadastradas as seis variáveis necessárias sem registrar valores no Git ou neste PDF; serviço em plano gratuito e região Virginia.

Próximo passo: revisar no Atlas a credencial e a lista de IPs permitidos, então confirmar /health com MongoDB ok. A fonte de requisitos foi o roteiro “Lab CICD-Pipeline.pdf”. Documentação consultada: render.com/docs/free, render.com/docs/deploys e render.com/docs/blueprint-spec.